DHT22 TEMPERATURE & HUMIDITY MONITORING SYSTEM

Repository Theory & Implementation Guide

Sensor fundamentals • Raspberry Pi GPIO • Python acquisition • validation • CSV logging • reliability • viva preparation

Repository: ApexHyper2007/DHT22-Temperature-Humidity-Monitoring-System

Prepared from the public README and main.py, with supporting technical references for the DHT22 sensor, Adafruit driver, and Raspberry Pi GPIO.

Scope note: this guide describes the current Raspberry Pi + Python implementation. The repository does not currently implement cloud connectivity or a web dashboard.

How to use this guide

This guide explains the theory behind the public repository **DHT22-Temperature-Humidity-Monitoring-System** and maps the concepts directly to its README and main.py. It is written to help you explain the design during a project review, understand the code, and debug common failures.

A useful reading order

- First pass: understand the system architecture and the sensor-to-file data path.
- Second pass: study GPIO wiring, DHT22 signaling, retries, validation, and CSV handling.
- Third pass: study errors, limitations, testing, and proposed extensions.
- Final pass: try answering the viva questions without looking at the answers.

Evidence and scope

Implementation-specific statements are based on the repository README and main.py. General sensor and GPIO explanations are marked as background theory. Future improvements are not described as existing features.

Important scope clarification: despite the repository About text using the phrase "Arduino-compatible microcontroller", the current code imports Adafruit_DHT, uses Raspberry Pi GPIO 18, and runs as a Python program. This guide describes the code that is actually present: a Raspberry Pi implementation, not an Arduino sketch.

Project objective and problem statement

The project monitors the surrounding environment by repeatedly reading temperature and relative humidity from a DHT22 digital sensor. Valid readings are printed to a terminal and saved locally for later inspection.

The problem being solved

A single reading gives only a momentary observation. Repeated readings create a time series: users can see whether a room is warming up, drying out, or experiencing changing humidity. Logging these observations makes later plotting and analysis possible.

Main features implemented

- Continuous reading loop, with a three-second delay between reading cycles.
- Use of the Adafruit_DHT DHT22 driver and its read_retry helper.
- Basic plausibility checks on temperature and humidity.
- Timestamped terminal output.
- Append-only historical CSV plus a file containing only the latest valid reading.
- Handling of common reading/logging exceptions and Ctrl+C shutdown.

What is outside the current scope

The repository does not currently provide a web dashboard, cloud database, remote access, alarms, or a charting interface. Its core deliverable is a local sensor acquisition and logging program.

End-to-end architecture

```
Physical environment
      | heat and moisture
      v
DHT22 sensor element
      | digital single-wire data signal
      v
Raspberry Pi GPIO 18 (physical header pin 12)
      |
      v
Adafruit_DHT.read_retry(...)
      |
      v
read_sensor(): check for missing / implausible values
      |
      +---- invalid ----> skip this cycle
      |
      v
print timestamp + temperature + humidity
      |
      v
save_reading()
      |
      +---- append ----> sensor_log.csv
      +---- overwrite -> current_reading.csv
```

This architecture separates sensing, interpretation, and persistence. The DHT22 creates measurements; the driver decodes the electrical protocol; Python validates the returned values; the loop presents valid values; CSV writes preserve the results.

Why the separation is useful

If values are missing, investigate wiring and sensor reads. If values are physically implausible, examine validation and environment. If terminal output appears but files do not update, investigate file paths and permissions. Locating the failure by stage is faster than treating the whole program as one black box.

IoT and edge monitoring concepts

The Internet of Things (IoT) refers to physical devices that sense or affect the world and exchange data with software systems. This project demonstrates the sensing and edge-processing parts of that pattern, even though it does not currently send data across a network.

Sensor, edge device, application

- Sensor: the DHT22 measures local temperature and relative humidity.
- Edge device: the Raspberry Pi powers the sensor, reads its data, runs Python, and writes records.
- Application/output: the terminal and local CSV files let a person inspect current and historical measurements.

Why local processing can be useful

Local processing can keep the system simple, work without internet access, and avoid recurring cloud dependencies. It also means the data may be lost if the storage device fails unless backups are configured.

What would make it a networked IoT monitor?

A future version could send validated measurements to a local broker or remote endpoint, store them in a database, and display charts. Network transport introduces new concerns: authentication, encryption, reconnect behavior, buffering during outages, and privacy. None of those network features should be claimed for the current repository.

DHT22 sensor theory

The DHT22, also sold as AM2302 in some forms, combines a capacitive humidity sensing element with a temperature sensor and on-board electronics. It reports measurements digitally over one data line. The application does not read an analog voltage and perform the sensor conversion itself; a compatible driver handles the digital protocol.

Temperature sensing

The temperature element changes an electrical property as temperature changes. The sensor electronics convert that signal into a numeric value and transmit it to the host. The Python program receives temperature in degrees Celsius.

Relative humidity sensing

A capacitive humidity element changes capacitance as water vapour affects its moisture-sensitive material. The sensor electronics convert that change into a relative-humidity value. Relative humidity is the amount of water vapour in air relative to the amount the air could hold at the same temperature, expressed as a percentage.

Digital output matters

The sensor packages conversion and communication into a digital reading. The host still needs correct timing, electrical connections, and a compatible driver. Digital output does not imply that readings can never fail or are perfectly accurate.

DHT22 operating range and cadence

The repository validates temperature from -40 to 80 degrees Celsius and humidity from 0 to 100 percent. Those bounds correspond to commonly documented DHT22 operating measurement ranges. A typical published temperature accuracy is around plus or minus 0.5 degrees Celsius, while humidity accuracy is often specified in a range such as 2-5 percentage points depending on the source and conditions.

Minimum reading interval

The DHT22 is relatively slow: a fresh reading should generally not be requested more frequently than about once every two seconds. The code sets READ_INTERVAL to three seconds, which is a sensible cadence for ordinary room monitoring. The read_retry function may itself perform retries and waiting, so the elapsed time between successful records can be longer than three seconds.

Resolution versus accuracy

Resolution describes the smallest reported step. Accuracy describes closeness to the true environmental value. A value printed to two decimal places does not mean the physical sensor is accurate to 0.01 degrees. The rounding mainly standardizes display and storage format.

The software range check is a plausibility check, not a calibration procedure. Passing the range check means a reading is numerically possible; it does not prove the sensor is correctly calibrated or located appropriately.

Hardware and electrical connections

The README specifies three core connections for the common DHT22 module: VCC to 3.3V, DATA to Raspberry Pi GPIO 18, and GND to ground. GPIO 18 is physical header pin 12 on the standard Raspberry Pi header layout. Confirm the header and sensor-module pin order before applying power.

Sensor connection	Repository configuration	Purpose
VCC	Raspberry Pi 3.3V	Powers the sensor module
DATA	GPIO 18 / physical pin 12	Carries the digital sensor signal
GND	Raspberry Pi GND	Provides a common electrical reference

Pull-up resistor

The DHT data line uses a single-wire signaling arrangement that commonly needs a pull-up resistor between DATA and the supply. Some breakout modules already include one. Check the specific module before adding a second resistor.

Electrical safety

Raspberry Pi GPIO uses 3.3V logic and its pins are not 5V tolerant. Do not drive a GPIO input directly from a 5V signal. Use the correct supply, share ground, and avoid moving wires while the circuit is powered.

If readings fail, check the sensor pin order, common ground, GPIO numbering scheme, data-line pull-up, power quality, and connection length before changing application code.

GPIO numbering: BCM versus physical pins

Raspberry Pi pin references often use two numbering systems. BCM numbering refers to the SoC GPIO identifier; physical numbering refers to the position on the board header. Confusing them can make the script read the wrong pin even if a wire is connected correctly.

This repository

The constant GPIO_PIN = 18 refers to BCM GPIO 18. The README correctly maps that signal to physical header pin 12.

```
Code constant:       GPIO_PIN = 18
BCM signal:          GPIO18
Physical header pin: 12
```

How to verify wiring

- Use the pinout diagram for the exact Raspberry Pi board.
- Check whether the library expects BCM pin numbers or physical pin numbers.
- Keep the code constant and the physical wiring consistent.
- Inspect the board pinout utility or trusted vendor documentation when unsure.

This project uses the legacy Adafruit_DHT interface, whose Raspberry Pi usage expects the GPIO number used by the library, not the physical header position. The value 18 in the code is therefore intentional for a wire on physical pin 12.

The DHT single-wire digital protocol

The DHT22 communicates using a timing-sensitive single data line. The host initiates a transaction, the sensor responds, and the bit values are represented by pulse timing. This is not the same protocol as I2C or SPI, and it is not the Dallas/Maxim 1-Wire protocol despite using one data wire.

Why precise timing matters

The host must distinguish low and high pulse durations correctly to decode the data. General-purpose operating systems such as Linux are not hard real-time systems, so scheduling delays can sometimes interfere with tight timing expectations. This is one reason libraries may return no reading occasionally.

Library abstraction

The application does not manually time pulses or decode individual bits. Adafruit_DHT.read_retry encapsulates the low-level interaction and retries a failed read according to the library's behavior. The program then handles the high-level values returned by the function.

Not like I2C

I2C normally uses separate SDA and SCL wires and supports addressed devices on a shared bus. DHT22's timing-based data line has different electrical and timing requirements. Do not assume generic I2C scanning tools can detect it.

Raspberry Pi, Python, and the runtime

The Raspberry Pi is a small Linux computer with GPIO hardware. Python runs as a user-space process and requests measurements through a library that interacts with GPIO timing and the sensor. The OS schedules the program and the process periodically sleeps between read attempts.

Why Python fits the task

Python provides straightforward file handling, formatted output, timestamps, exception handling, and access to sensor libraries. These capabilities are sufficient for a local environmental logger without needing a database server or web stack.

Files and working directory

The code writes sensor_log.csv and current_reading.csv using relative paths. A relative path is resolved against the process's current working directory, which may differ depending on how the program is launched. Run the script from the intended project folder or change the paths to explicit locations if predictable output placement is important.

Permissions

The process needs permission to access the GPIO subsystem and write to its current directory. Running everything as root may mask permission problems but is not always the best operational choice; prefer a supported library installation and a least-privilege configuration.

The Adafruit_DHT library

The source imports Adafruit_DHT and sets SENSOR = Adafruit_DHT.DHT22. The driver provides the sensor type constant and the read_retry function. This keeps protocol-specific handling out of the application's business logic.

What read_retry returns

The function returns two values in this order: humidity, temperature. The code then checks whether either is None and, if both are present, returns temperature first and humidity second from its own helper. Remember this order change when debugging or reusing the function.

```
humidity, temperature = Adafruit_DHT.read_retry(SENSOR, GPIO_PIN)
# application helper returns: (temperature, humidity)
```

Dependency maintenance note

The upstream Adafruit_Python_DHT repository is archived and marked deprecated. The current project uses that legacy API. For a fresh setup on newer Raspberry Pi OS/Python versions, check current Adafruit-supported CircuitPython DHT guidance and test compatibility before assuming the old installation command will work unchanged.

Theory versus current implementation: the existing main.py uses Adafruit_DHT.read_retry. A migration to adafruit-circuitpython-dht would require code and environment changes; it is not a drop-in feature that this repository already implements.

Configuration constants in main.py

The first part of main.py collects key configuration values. Centralizing them makes it easier to adjust hardware or output behavior without rewriting the monitoring loop.

Constant	Current value	Meaning
SENSOR	Adafruit_DHT.DHT22	Driver sensor type
GPIO_PIN	18	BCM GPIO connected to DATA
READ_INTERVAL	3	Seconds slept between loop cycles
LOG_FILE	sensor_log.csv	Historical append-only data file
CURRENT_FILE	current_reading.csv	Latest valid reading snapshot
CSV_FIELDS	timestamp, temperature_c, humidity_percent	CSV column order

Why constants help

Constants communicate assumptions to a reader and reduce duplicated magic values. For an expanded project, configuration might move into a command-line parser or config file, but for a short script these constants are easy to understand.

Change carefully

Changing GPIO_PIN requires matching physical rewiring. Changing READ_INTERVAL should respect the sensor's sampling cadence and retry time. Changing filenames affects where downstream analysis tools look for data. Changing CSV_FIELDS changes the output schema and may break scripts that consume the existing files.

Function walkthrough: read_sensor()

read_sensor() is the boundary between low-level sensor acquisition and the rest of the program. It reads the sensor, checks whether the returned values exist, checks broad valid ranges, rounds successful values, and returns a pair.

```
def read_sensor():
    humidity, temperature = Adafruit_DHT.read_retry(SENSOR, GPIO_PIN)
    if humidity is None or temperature is None:
        print("Sensor reading failed; will retry.")
        return None
    if not (-40 <= temperature <= 80 and 0 <= humidity <= 100):
        print("Invalid sensor reading; skipping.")
        return None
    return round(temperature, 2), round(humidity, 2)
```

Why return None on failure?

A None return is a simple signal that no acceptable reading is available. The main loop can then skip printing and writing a record while continuing the next cycle.

Why return a tuple?

Returning (temperature, humidity) groups related values into one result while preserving their order. The caller unpacks the tuple into two variable names.

This helper makes the program easier to reason about because saving code sees only values that have already passed the basic checks. A stronger design might use a structured object with status metadata, but the tuple is sufficient for this small script.

Retries, missing values, and robustness

A digital sensor may occasionally fail to provide a decodable measurement because of timing, power, wiring, or environmental conditions. The library's read_retry function performs its own retry attempts. If no valid result is returned, it can provide None values to the caller.

Two levels of control

- Library-level retry: tries to obtain a sensor measurement during a single function call.
- Application-level loop: after the call returns, the script continues monitoring after its configured sleep.

Why not write failed reads as zero?

Zero degrees Celsius and zero percent humidity can be legitimate values under some conditions. Replacing missing readings with zero would corrupt the dataset and make a communication failure appear to be a real environmental observation.

Avoid retry storms

A retry strategy should have bounded attempts and appropriate delays. The project relies on read_retry and then sleeps for READ_INTERVAL, which is simple. More advanced monitoring could track consecutive failures, report uptime, and back off when the sensor is disconnected.

A skipped reading is not the same as a zero reading. Keeping this distinction in the data is essential for valid analysis.

Input validation and plausibility checks

The code rejects temperature values outside -40 to 80 degrees Celsius and humidity values outside 0 to 100 percent. These are broad domain bounds intended to discard impossible or malformed values before recording them.

Why validate at the application layer?

Libraries decode the protocol, but application code still benefits from checking domain assumptions. A value can be syntactically numeric and still be invalid for the sensor. Validation protects the historical log from obvious outliers.

What validation does not catch

- A sensor that is consistently biased but remains in range.
- A sensor placed near a heater that reports the local microclimate rather than room average.
- A sensor whose readings are stale but numerically plausible.
- A sensor wired correctly but affected by condensation, self-heating, or poor ventilation.

Possible stronger checks

A larger system could detect sudden jumps, repeated identical values, long periods without successful readings, or implausible rates of change. Such filters must be chosen carefully because real environments can change quickly too. Preserve raw readings when possible so validation rules can be reviewed later.

Timestamps and datetime

save_reading() calls datetime.now().isoformat(timespec="seconds") and stores the result in the timestamp field. The timestamp allows a value to be associated with the time at which the script recorded it.

Why ISO-style timestamps?

An ISO-like timestamp such as 2026-10-20T18:30:00 is compact, sortable as text in common cases, and widely recognized by software tools. The code uses local system time and second precision.

Local time versus UTC

datetime.now() produces local system time without attaching a timezone offset. If the file is analyzed on another machine or merged with data from multiple devices, timezone ambiguity can become a problem. A future distributed system should store UTC timestamps or include the offset, while separately displaying local time for humans.

Measurement time caveat

The recorded timestamp is created after a successful sensor read is returned. It is therefore close to the recording time, not a hardware-provided timestamp embedded in the sensor measurement.

If the Raspberry Pi clock is wrong, logs may be out of order or have incorrect wall-clock timestamps. For long-running unattended use, maintain the system clock using an appropriate time synchronization mechanism.

CSV logging: structure and benefits

CSV stands for comma-separated values. It stores table-like records in a plain-text file and can be read by spreadsheet software or Python data-analysis libraries. This project uses csv.DictWriter so each field is associated with a named column.

Columns

```
timestamp,temperature_c,humidity_percent
2026-10-20T18:30:00,28.14,61.70
2026-10-20T18:30:03,28.16,61.65
```

The example values above are illustrative, not readings claimed to have come from the user's sensor. The actual script rounds measurements to two decimal places before saving them.

Why use DictWriter?

DictWriter makes the intended output schema explicit. The CSV_FIELDS list controls header names and column ordering. The reading dictionary maps timestamp, temperature_c, and humidity_percent to their values.

CSV trade-offs

CSV is transparent, compact, and easy to inspect. It does not provide transactions, concurrent-writer coordination, schema enforcement, or efficient indexed queries like a database. For a single local process and small logs, CSV is a reasonable starting point.

Historical log versus latest snapshot

The script writes every successful reading to two files, each with a different purpose. This is an intentional small design pattern: one file preserves history, while another offers a simple current-state view.

sensor_log.csv

write_reading(LOG_FILE, reading, append=True) opens the historical file in append mode. Existing records remain, and one new row is added for each successful observation. The header is written only when the file is new or empty.

current_reading.csv

write_reading(CURRENT_FILE, reading, append=False) opens the current file in write mode. Each new valid reading replaces the previous contents, leaving a header and one latest row.

File	Open mode	Expected contents
sensor_log.csv	a (append)	Header + all successful records
current_reading.csv	w (overwrite)	Header + latest successful record

Operational consequences

Invalid reads do not update either file. After shutdown, current_reading.csv therefore contains the last valid reading, not a failure status. The historical log grows for as long as the program runs; a long-term system may need log rotation or archiving.

The main loop and scheduling

main() prints a banner and configuration summary, then enters a while True loop. Each cycle calls read_sensor(), prints and saves only when a valid tuple is returned, and then sleeps for READ_INTERVAL seconds.

```
while True:
    try:
        result = read_sensor()
        if result is not None:
            temperature, humidity = result
            print(...)
            save_reading(temperature, humidity)
    except (OSError, RuntimeError, csv.Error) as error:
        print(f"Reading or logging error: {error}")
    time.sleep(READ_INTERVAL)
```

Why sleep?

Without the sleep, the program would request reads as quickly as the CPU and driver allow, wasting resources and potentially asking the DHT22 for data too frequently. The three-second pause keeps routine polling restrained.

Actual period is not exactly three seconds

The delay occurs after the read and logging work. If a read takes 0.4 seconds and writing takes 0.1 seconds, the next loop begins around 3.5 seconds later. Internal retries can lengthen it further. If exact sampling times become important, use a monotonic clock and schedule next deadlines rather than simply sleeping after each cycle.

Error handling: exceptions versus bad data

The project has two distinct error paths. Bad or missing sensor values are handled as data-quality outcomes inside read_sensor(). Exceptions such as OSError, RuntimeError, or csv.Error are caught by the loop and printed as runtime problems.

Data failure

A None value or an out-of-range value results in a message and a return of None. The main loop skips the record and carries on.

Operational exception

A GPIO or I/O problem may raise an exception rather than return a bad measurement. The outer loop catches a selected set of exceptions so one recoverable failure need not terminate the process.

Why catch specific exceptions?

Specific exception lists document expected failures and avoid silently swallowing every possible bug. Catching Exception indiscriminately can make programming errors hard to notice. The trade-off is that an unexpected exception not in the list can still terminate the process; robust applications log a traceback and define a deliberate recovery policy.

The sleep call is outside the inner try block but inside the while loop and outer KeyboardInterrupt handler. A failure in sleep itself is unusual; Ctrl+C is handled at the outer level.

Graceful shutdown and Ctrl+C

The main function wraps the infinite loop in try/except KeyboardInterrupt. When the user presses Ctrl+C in the terminal, Python typically raises KeyboardInterrupt. The handler prints a shutdown message instead of showing a full traceback.

Why it matters

A readable shutdown makes manual operation friendlier. The program uses short-lived CSV file contexts with the with statement, so files are closed after each write even when the loop later stops. The script does not currently need a long-lived database connection or a separate cleanup routine.

What graceful shutdown does not mean

Ctrl+C handling does not guarantee that every measurement has been written during power loss or sudden process termination. Each row is written through normal file operations, but reliability requirements such as power-failure durability, buffered writes, and filesystem behavior need separate design choices.

Potential unattended operation

If deployed as a background service, configure a service manager to restart after faults, capture standard output and errors, and stop the process cleanly. The repository currently documents interactive Ctrl+C usage rather than a service configuration.

Detailed source walkthrough: from imports to main()

Imports

csv provides CSV readers/writers; os checks whether a file exists and has content; time provides sleep; datetime creates timestamps; Adafruit_DHT provides sensor constants and acquisition.

read_sensor()

Obtains humidity and temperature in the driver's return order, rejects None values, enforces broad limits, rounds both values, and returns temperature first.

write_reading()

Checks whether a non-empty file exists, chooses append or overwrite mode, creates a DictWriter with the configured fields, writes a header when needed, and writes one row.

save_reading()

Creates a dictionary containing timestamp, temperature_c, and humidity_percent, then writes it to the historical log and current snapshot.

main()

Displays startup information, repeats acquisition, prints valid observations in a readable format, persists the record, handles expected runtime exceptions, sleeps, and exits cleanly on KeyboardInterrupt.

The if __name__ == "__main__": guard means main() runs when this file is executed directly. Importing main.py from another Python module does not automatically start the infinite monitoring loop.

Understanding sample console output

A successful iteration prints the current local timestamp followed by temperature in degrees Celsius and humidity as a percentage. The format specifier :.2f ensures exactly two decimal places are displayed.

```
2026-10-20 18:30:00 | Temperature: 28.14 °C | Humidity: 61.70%
Sensor reading failed; will retry.
Invalid sensor reading; skipping.
Reading or logging error: [error details]
Monitoring stopped.
```

These lines illustrate possible message formats. They are examples for learning, not captured output from a physical device.

How to interpret the messages

- A valid line means the returned values passed the current range checks and were then sent to the saving function.
- A reading-failed message generally points to a failed sensor transaction or unavailable result.
- An invalid-reading message means values were returned but outside the application's allowed ranges.
- A logging-error message points to an exception from the read/logging area caught by the loop.
- Monitoring stopped means KeyboardInterrupt was handled.

If the success message prints but the files appear missing, check the program's current working directory. Relative paths can place CSV files somewhere other than the repository folder if the script was launched from a different directory.

Installation and run workflow

The README says to run the application with python3 main.py on a compatible Raspberry Pi OS setup after installing the sensor library. Physical GPIO access is required; a standard Windows laptop cannot directly read Raspberry Pi GPIO 18.

Practical setup checklist

- Confirm the Raspberry Pi model and OS, then inspect the correct GPIO header pinout.
- Connect sensor VCC, DATA, and GND with power removed.
- Install a DHT library version compatible with the OS and Python version.
- Run the program from the intended project directory.
- Check terminal output and confirm both CSV files update after successful readings.

README cleanup item

The published README's clone command points to https://github.com/TabletopPants29/DHT22.git and tells the reader to enter the DHT22 directory. That URL does not match this repository's URL. For a user cloning this project, the command should point to ApexHyper2007/DHT22-Temperature-Humidity-Monitoring-System and use the matching folder name.

Legacy library warning

Adafruit's original Adafruit_Python_DHT repository is archived. New setups may need a maintained compatible alternative or a documented environment that supports the existing API. Resolve this compatibility question early, because the application cannot read hardware if the imported library is unavailable.

Troubleshooting decision tree

No reading ever succeeds

- Check the physical sensor pin order and common ground.
- Verify GPIO 18 means BCM GPIO 18 in the installed library and is wired to physical pin 12.
- Confirm the module is powered at a supported voltage and that the data line has an appropriate pull-up.
- Check library installation, OS compatibility, and permissions.

Readings fail intermittently

- Keep read intervals at or above the sensor's minimum sampling cadence.
- Shorten or improve noisy/loose wiring and stabilize sensor power.
- Consider OS timing variability and use a compatible driver.

CSV files are not found

- Print os.getcwd() or use absolute paths to locate the runtime directory.
- Check directory permissions and disk space.
- Verify the latest record is only updated after a valid read.

Wrong-looking measurements

- Compare against a known reference thermometer/hygrometer over a stable period.
- Keep the sensor away from direct sun, heat-generating boards, vents, and condensation.
- Remember that two decimal display places are formatting, not accuracy.

Debug in order: electrical setup -> driver import -> raw sensor return -> validation -> console output -> file writing. Avoid changing several layers at once.

Data quality and environmental measurement

Environmental sensors produce observations with uncertainty. A measurement is affected by sensor accuracy, placement, response time, calibration, airflow, local heating, humidity exposure, and timing. Software can reject obviously impossible readings, but it cannot eliminate physical measurement error.

Placement

Place the sensor in representative air and allow it to equilibrate. Avoid touching the sensing area, enclosing it without airflow, or positioning it directly above warm electronics. Condensation and moisture can damage some modules or distort measurements.

Calibration and comparison

For a simple project, compare the sensor against a trusted reference in the same environment and wait for readings to stabilize. Record the difference. A formal calibration requires traceable standards and a defined procedure; merely observing one matching reading is not proof of calibration.

Outliers and trends

Historical readings are valuable because trends can reveal sensor dropouts or environment changes. A rolling average can reduce random fluctuations but also hides rapid changes and adds lag. Keep raw measurements when possible and clearly label any filtered values.

The current program stores only accepted measurements; it does not separately log invalid raw values or failure counters. That makes the log cleaner, but it also limits later analysis of sensor reliability.

What the project does not yet do

A clear explanation of limitations is part of sound engineering. The README states that data is stored locally and there is no cloud storage or remote monitoring. The current code also does not implement graphs, alerts, a web dashboard, a database, or automated long-term log rotation.

Practical implications

- There is no remote view when the operator is away from the Raspberry Pi.
- If the device loses power, the script stops until restarted.
- Historical CSV data can grow without bound.
- The latest-reading file contains the last valid reading, which may be old if the sensor stops responding.
- No watchdog currently tells a user how long it has been since a successful read.

Document current reality honestly

The project is a local environmental sensing and logging prototype. It is a useful basis for learning hardware-software interfacing, data validation, and persistence. Do not describe it as a cloud IoT platform or real-time dashboard unless those functions are actually added and tested.

Reliability, safety, and maintainability

Reliability

Reliability can be improved by tracking consecutive sensor failures, recording explicit health status, adding automated restarts, and testing behavior during unplug/replug events. Each improvement should define what the expected system behavior is before implementation.

Data resilience

Back up the CSV file if measurements matter. Consider rotating logs daily or by size, writing into a dedicated data folder, and detecting disk-full errors. If multiple programs write to the same file, introduce synchronization or move to a database.

Security

The current program has no network endpoint, which limits its network attack surface. If remote functionality is added, avoid exposing unauthenticated services directly to the public internet, protect credentials, and use encrypted transport.

Maintainability

Keep configuration and data schemas documented, add tests for helper functions, and avoid undocumented dependency assumptions. A concise README should provide correct clone instructions, compatible library installation notes, wiring details, and expected output files.

Safety remains a hardware concern: check voltage levels and pin assignments before powering the circuit. GPIO damage can be caused by incorrect connections even if the Python code itself is correct.

Testing strategy

Hardware-dependent applications need both software tests and real-device tests. Unit tests can verify file-writing rules without a sensor; hardware smoke tests confirm actual wiring, driver behavior, and physical measurements.

Unit-test ideas

- Mock read_retry to return valid values, None, and out-of-range values; verify read_sensor behavior.
- Test write_reading on a temporary directory: header once, append rows, overwrite snapshot.
- Test save_reading's expected column names and timestamp format.
- Test exceptions from file opening/writing and confirm the loop reports the failure.

Hardware smoke tests

- Run for several minutes and confirm that valid records appear with timestamps.
- Disconnect the data line briefly and verify failures are skipped, not recorded as zeros.
- Restore the connection and verify monitoring resumes.
- Press Ctrl+C and verify the process exits without a noisy traceback.

Acceptance criteria

Define observable outcomes: valid records have three fields; history grows by one row per success; current snapshot has one data row; missing measurements are skipped; the process can be stopped interactively. These criteria turn the README's feature claims into testable behavior.

Possible improvements and extension roadmap

Stage 1: strengthen the current logger

- Use pathlib for explicit data paths and create a dedicated data directory.
- Add structured logging, a failure counter, and last-success timestamp.
- Add command-line options for GPIO pin, interval, and output directory.
- Pin and document tested dependency versions.

Stage 2: data analysis

- Add a Python script to plot temperature and humidity against time.
- Calculate daily minimum, maximum, mean, and missing-reading percentage.
- Detect long gaps or sudden jumps without discarding original measurements.

Stage 3: remote view and alerts

- Send validated readings to an MQTT broker or a secured HTTP endpoint.
- Store observations in a database or time-series service.
- Build a dashboard and configurable threshold alerts.

Trade-offs

Each extension increases dependency and operational complexity. Keep measurement and local logging reliable first; then add network transport and visualization as separate modules. A modular design lets the sensor logger continue working during a network outage.

Viva questions: sensor and hardware

1. What is the purpose of the project?

To read temperature and relative humidity from a DHT22 on a Raspberry Pi, show valid readings, and save timestamped data locally.

2. What does DHT22 measure?

Ambient temperature and relative humidity, returned as digital values.

3. Why is it called a digital sensor?

Its on-board electronics convert measurements and transmit encoded digital data instead of providing a raw analog output to be digitized by the Pi.

4. Which GPIO is used?

BCM GPIO 18, which is physical header pin 12 on the standard Raspberry Pi header.

5. Why is ground shared?

A common electrical reference is required for the data signal to be interpreted correctly.

6. Why may a pull-up resistor be needed?

The single data line needs a defined high level when no device is pulling the line low; some modules include the resistor already.

7. Why not read the sensor every millisecond?

The DHT22 cannot provide fresh measurements that quickly. Its typical minimum sampling interval is around two seconds, and too-frequent requests can return stale or failed reads.

Viva questions: code and data logging

8. What is read_retry doing?

It asks the DHT driver to attempt the sensor transaction again when a read fails. The application still checks for missing return values.

9. What does None mean here?

The library did not provide a usable measurement for that attempt. The program skips saving it.

10. Why are the two returned values unpacked as humidity, temperature?

That is the order used by Adafruit_DHT.read_retry. The helper deliberately returns them in the opposite order: temperature, humidity.

11. What is the purpose of range checks?

To reject values outside broad physically plausible ranges for the sensor.

12. What is CSV?

A plain-text format for table-like records, where fields are separated by commas and rows represent records.

13. Why two CSV files?

sensor_log.csv preserves history by appending; current_reading.csv is overwritten to contain only the most recent valid reading.

14. Why use datetime.now()?

To associate each accepted observation with a local timestamp.

15. What does Ctrl+C do?

It raises KeyboardInterrupt in the terminal process; the handler prints a shutdown message and exits the monitoring loop.

Viva questions: errors and design decisions

16. Does a range-valid value guarantee accuracy?

No. A sensor can be biased or badly placed and still report a value within the valid range.

17. Why is READ_INTERVAL set to three seconds?

It provides a modest delay above the DHT22's typical two-second minimum read interval. Actual time between records also includes read and write duration.

18. What if the latest read fails?

Neither file is updated, so current_reading.csv continues to hold the last valid reading.

19. Is this currently a cloud IoT system?

No. The current implementation reads a physical sensor and stores the output locally. Cloud storage and remote monitoring are listed as future enhancements.

20. Why can Raspberry Pi timing affect readings?

Linux is not a hard real-time operating system, and scheduling jitter can disrupt timing-sensitive single-wire decoding.

21. What is one documentation issue?

The README's clone command points to a different GitHub repository instead of this repository.

22. What dependency concern should be investigated?

The original Adafruit_Python_DHT package is archived/deprecated, so compatibility with a newer OS/Python environment should be checked.

23. What would you improve first?

Correct the clone instructions, document a verified installation path, add software tests for validation and file writes, and expose the most recent successful reading time.

Quick reference: files, functions, and variables

Name	Role
main.py	The complete monitoring script in this repository.
read_sensor()	Read the DHT22 and reject missing or out-of-range values.
write_reading()	Write one dictionary record to a CSV file using append or overwrite mode.
save_reading()	Create a timestamped record and update both CSV destinations.
main()	Run the infinite polling loop, print valid measurements, handle expected errors.
SENSOR	Select Adafruit_DHT.DHT22.
GPIO_PIN	BCM GPIO 18.
READ_INTERVAL	Three-second post-cycle sleep.
sensor_log.csv	Historical data log.
current_reading.csv	Latest valid record snapshot.

Commands in the README

```
python3 main.py
# Stop interactively with Ctrl+C
```

Official repository links

Project README:
https://github.com/ApexHyper2007/DHT22-Temperature-Humidity-Monitoring-System/blob/main/README.md

Source code: https://github.com/ApexHyper2007/DHT22-Temperature-Humidity-Monitoring-System/blob/main/main.py

Use these links when checking whether implementation details have changed since this guide was written.

References and final understanding check

Primary source code and project-specific claims:

- ApexHyper2007, DHT22 Temperature & Humidity Monitoring System README: https://github.com/ApexHyper2007/DHT22-Temperature-Humidity-Monitoring-System/blob/main/README.md
- ApexHyper2007, main.py: https://github.com/ApexHyper2007/DHT22-Temperature-Humidity-Monitoring-System/blob/main/main.py

Technical background:

- Adafruit DHT22 / AM2302 product and specifications: https://www.adafruit.com/product/385
- Adafruit DHT sensor overview: https://learn.adafruit.com/dht
- Legacy Adafruit Python DHT repository and maintenance status: https://github.com/adafruit/Adafruit_Python_DHT
- Adafruit CircuitPython DHT library: https://github.com/adafruit/Adafruit_CircuitPython_DHT
- Raspberry Pi hardware and GPIO documentation: https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

Can you explain the whole data path?

The DHT22 senses temperature and humidity and transmits digital data on its data line. Python calls the Adafruit driver on BCM GPIO 18. read_sensor checks that both values exist and fall within permitted ranges. main prints accepted values and save_reading creates a timestamped dictionary. write_reading appends it to the historical log and overwrites the latest-reading snapshot. Failures are skipped or reported, and Ctrl+C stops the loop.

If you can explain that paragraph in your own words, describe why each check exists, and troubleshoot a missing reading in a systematic order, you understand the central design of this repository.